

Dokumentation: Automatisierungsprojekt zur Systembereinigung (Modul 122)

Inhalt

1. Einleitung	2
2. Fachliche Beschreibung / Businessanalyse	2
3. Zieldefinition (Muss/Kann)	2
4. Design / Programmablaufplan (PAP)	3
5. Script Umsetzung	4
6. Automatisierung	5
7. Tests / Testqualität	6
8. Idee und Kreativität	6
9. Herausforderungen und Fehlerbehebung.....	7

1. Einleitung

Im Rahmen des Moduls 122 wurde ein Projekt zur Systemautomatisierung realisiert. Ziel des Projekts ist es, wiederkehrende Wartungsaufgaben auf einem Windows-System mittels eines Bash-Skripts zu automatisieren. Die manuelle Bereinigung von temporären Dateien und dem Papierkorb wird oft vernachlässigt, was langfristig zu Speicherplatzmangel und Systemträgheit führen kann. Dieses Skript bietet eine effiziente Lösung, um diese Aufgaben zeitgesteuert und zuverlässig durchzuführen.

2. Fachliche Beschreibung / Businessanalyse

In einer produktiven Arbeitsumgebung sammeln sich täglich grosse Mengen an temporären Daten an (Cache, Installationsreste, temporäre User-Files). Zudem belegt der Papierkorb oft Gigabytes an Speicherplatz, die dem System nicht zur Verfügung stehen. Die Businessanalyse ergab folgende Anforderungen:

- **Effizienz:** Reduzierung des manuellen Aufwands für Systemwartung.
- **Transparenz:** Überwachung des aktuellen Speicherplatzes.
- **Sicherheit:** Automatisierte Protokollierung aller durchgeführten Aktionen zur Nachvollziehbarkeit.

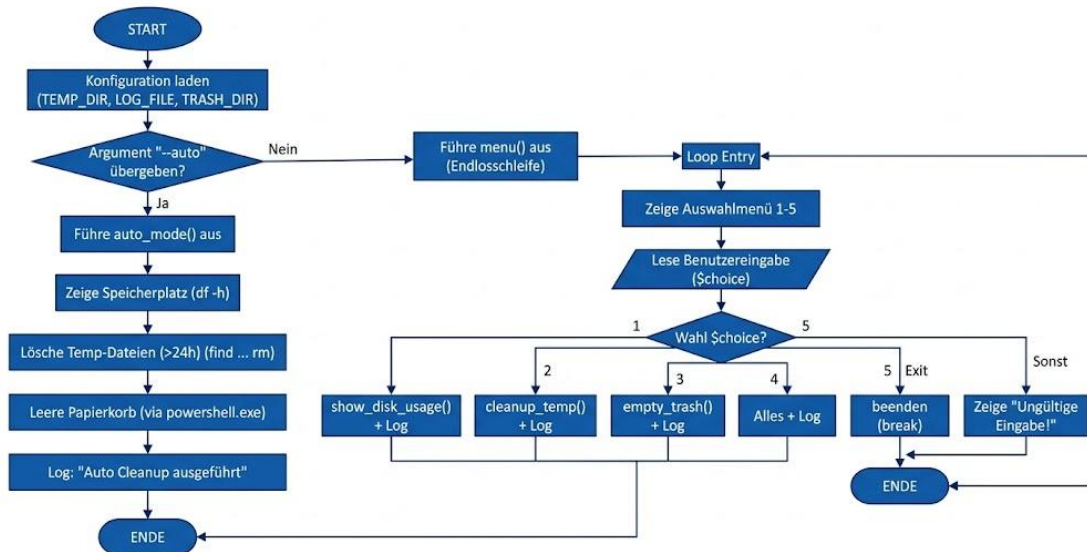
3. Zieldefinition (Muss/Kann)

Kategorie	Ziel	Status
Muss-Ziel	Automatisierte Leerung des Papierkorbs	Abgedeckt
Muss-Ziel	Löschen von Temp-Dateien (älter als 24h)	Abgedeckt
Muss-Ziel	Anzeige der aktuellen Speicherauslastung	Abgedeckt
Muss-Ziel	Erstellung eines Menüs zur manuellen Steuerung	Abgedeckt
Muss-Ziel	Automatisierung via "Cron"-Logik (Task Scheduler)	Abgedeckt
Kann-Ziel	Detailliertes Logging in eine externe Datei	Abgedeckt

4. Design / Programmablaufplan (PAP)

Das Skript folgt einer klaren Logik: Beim Start wird geprüft, ob ein Parameter übergeben wurde. Falls nicht, startet das interaktive Menü. Falls der Parameter --auto übergeben wird, führt das Skript alle Funktionen ohne Benutzereingabe aus.

PROGRAMMABLAUFPLAN: BASH CLEANUP SKRIPT (cleanup.sh)



5. Script Umsetzung

Das Skript wurde modular in Bash aufgebaut, um eine klare Trennung zwischen Konfiguration, wiederverwendbaren Funktionen und der eigentlichen Ausführungslogik zu gewährleisten. Anstatt den gesamten Code sequenziell ablaufen zu lassen, reagiert das Programm dynamisch auf die Art des Aufrufs.

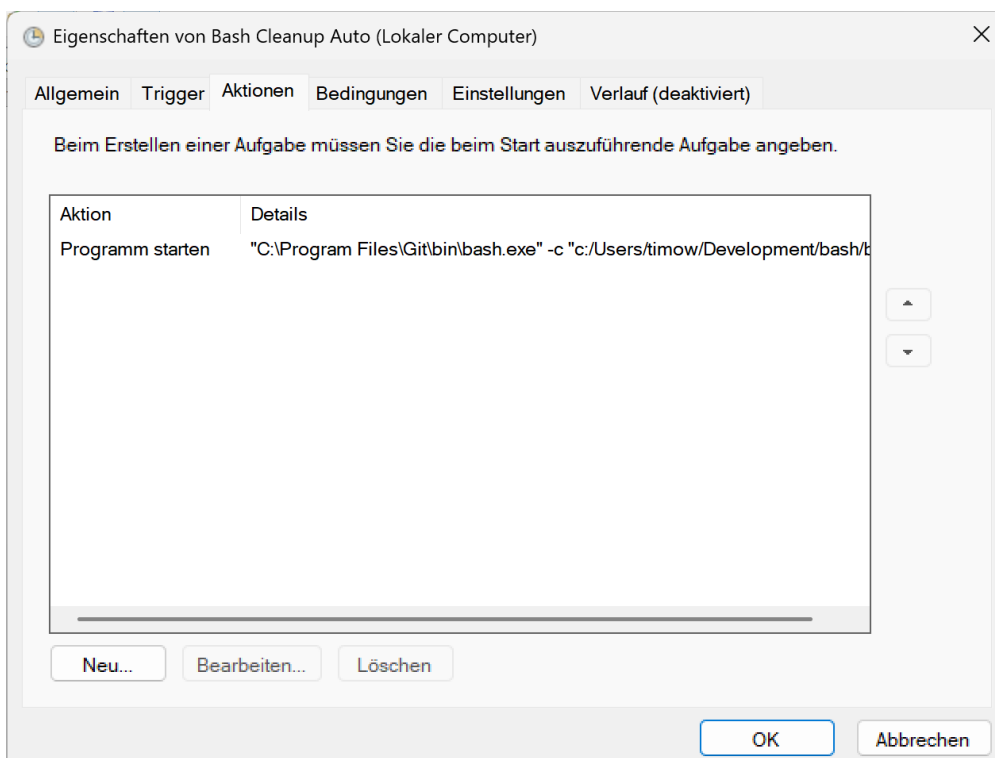
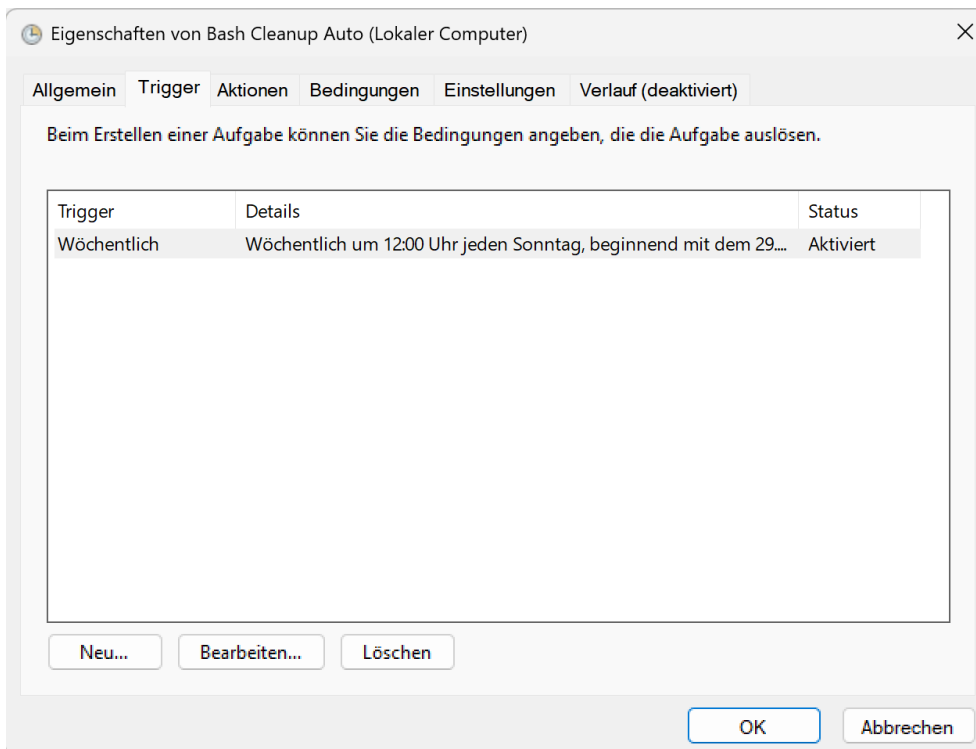
Die technische Umsetzung gliedert sich in folgende Kernbereiche:

- **Konfiguration & Variablen:** Zu Beginn des Skripts werden die absoluten Pfade für das temporäre Verzeichnis, den Windows-Papierkorb und die zentrale Log-Datei in Variablen definiert. Dies erleichtert zukünftige Anpassungen, da Pfade nur an einer Stelle geändert werden müssen.
- **Modularer Aufbau (Funktionen):** Jede Hauptaufgabe wurde in eine eigene Funktion ausgelagert (z. B. `show_disk_usage`, `cleanup_temp`, `empty_trash`). Dies hält den Hauptcode sauber. Besonders hervorzuheben ist die `empty_trash`-Funktion: Da die reine Bash unter Windows den echten System-Papierkorb nicht zuverlässig leeren kann, greift das Skript hier über eine Schnittstelle auf die Windows PowerShell (`powershell.exe -Command ...`) zu.
- **Interaktives Benutzermenü (Schleifen und Bedingungen):** Für den manuellen Betrieb wurde eine Endlosschleife (`while true`) implementiert, die dem Benutzer ein Menü (Optionen 1 bis 5) anzeigt. Die Benutzereingabe wird eingelesen und durch eine umfassende `if-elif-else`-Kontrollstruktur ausgewertet. Je nach Auswahl werden die entsprechenden Funktionen aufgerufen oder die Schleife wird mit `break` (Option 5) sauber beendet.
- **Automatisierungslogik (Auto-Modus):** Am Ende des Skripts entscheidet eine finale `if`-Abfrage über den Betriebsmodus. Wird das Skript mit dem Argument `--auto` aufgerufen, überspringt es das Benutzermenü vollständig. Stattdessen werden alle Reinigungsfunktionen und die Festplattenauswertung sequenziell im Hintergrund ausgeführt.
- **Protokollierung (Logging):** Eine separate Logging-Funktion sorgt dafür, dass jede durchgeführte Aktion (sowohl im manuellen als auch im automatischen Modus) mit einem aktuellen Zeitstempel in eine externe Datei (`cleanup.log`) geschrieben wird. Dies gewährleistet die nachträgliche Kontrolle der automatisierten Tasks.

6. Automatisierung

Da Git Bash unter Windows keinen nativen Cron-Daemon besitzt, wurde die Automatisierung über die Windows Aufgabenplanung realisiert. Die Konfiguration entspricht der Cron-Syntax `0 12 * * 0` (Jeden Sonntag um 12:00 Uhr).

Das Skript wird mit dem Argument `--auto` aufgerufen, was eine Ausführung ohne Benutzereingabe ermöglicht.



7. Tests / Testqualität

Testfall	Eingabe / Aktion	Erwartetes Ergebnis	Status
Manueller Test	Auswahl "1" im Menü	Anzeige der Festplattenbelegung	Bestanden
Temp-Bereinigung	Auswahl "2" im Menü	Löschen alter Dateien im Temp-Verzeichnis	Bestanden
Papierkorb leeren	Auswahl "3" im Menü	Löschen der Dateien im Papierkorb	Bestanden
Alles ausführen	Auswahl "4" im Menü	Führt alle Befehle aus (Anzeige des Speicherplatzes, temp-Bereinigung, Leerung des Papierkorbs)	Bestanden
Auto-Modus	Aufruf via <code>./cleanup.sh --auto</code>	Skript läuft ohne Interaktion durch und schreibt Log	Bestanden
Logging	Nach Ausführung	Eintrag in <code>./logs/cleanup.log</code> vorhanden	Bestanden

Alle Tests der Tabelle wurden auf einem Windows 11 System, mit einem Benutzer, der über Admin Rechte verfügt durchgeführt.

8. Idee und Kreativität

Die Besonderheit dieses Projekts liegt in der Brückenbildung zwischen Unix-Tools (Bash, find, df) und Windows-Systemfunktionen (PowerShell, Task Scheduler). Durch die Implementierung eines Argument-Parsers (`--auto`) ist das Skript universell einsetzbar – sowohl für den Administrator am Terminal als auch für das System im Hintergrund.

9. Herausforderungen und Fehlerbehebung

Während der Entwicklung des Skripts traten einige plattformspezifische Herausforderungen auf, insbesondere bei der Leerung des Papierkorbs. Da das Skript in einer Git Bash (Linux-Emulation) auf einem Windows-System läuft, unterschieden sich die Pfadstrukturen gravierend.

Das Problem: Anfänglich wurde versucht, den Papierkorb über den typischen Linux-Pfad zu leeren (z. B. `rm -rf ~/.local/share/Trash/files/*`). Dieser Befehl lief zwar ohne Fehlermeldung durch, leerte jedoch nicht den echten Windows-Papierkorb (`$Recycle.Bin`), sondern nur einen von der Bash simulierten Ordner. Der Speicherplatz auf der Festplatte wurde dadurch nicht freigegeben.

Die Lösung: Nach einigen Recherchen und Tests musste der Code für diese spezifische Funktion umgebaut werden. Die Lösung bestand darin, aus dem Bash-Skript heraus direkt das Windows-System anzusprechen. Dies wurde realisiert, indem die Windows PowerShell aufgerufen und ihr der Befehl `Clear-RecycleBin` übergeben wurde. Um zu verhindern, dass das Skript bei einem bereits leeren Papierkorb abbricht oder auf eine Bestätigung des Users wartet, wurden die Parameter `-Force` und `-ErrorAction SilentlyContinue` ergänzt. Dieser Umbau war entscheidend, um das Skript für eine Windows-Umgebung voll funktionsfähig zu machen.